

Uso de Inteligencia Artificial en el Desarrollo del Proyecto Mochila Antirrobo con IoT

Alvarez Santiago Javier, Orellano Nazareno Asaf, Raimo Lugo Kevin, Altamirano
Fabrizio Augusto, Da Silva Lautaro Nahuel
45738927, 42562943, 43915535, 44161995, 42816815
Lunes, 4

¹Universidad Nacional de La Matanza,
Departamento de Ingeniería e Investigaciones Tecnológicas,
Florencio Varela 1903 - San Justo, Argentina

Resumen. El presente informe documenta el proceso de desarrollo asistido por Inteligencia Artificial (IA) del proyecto, contrastando una implementación realizada de forma manual con una implementación obtenida mediante técnicas de "vibecoding". Se detalla la herramienta y el modelo de IA utilizados, los prompts técnicos empleados, la cantidad de tokens consumidos, los tiempos requeridos para obtener una versión funcional y las métricas de uso de CPU y memoria de cada enfoque. Asimismo, se presenta el repositorio de código versionado para ambas implementaciones y se contrasta el resultado obtenido contra la rúbrica de funcionalidad establecida, para la aplicación desarrollada en Android para la mochila Antirrobo con IoT.

Palabras claves: Inteligencia Artificial, Vibecoding, Android, IoT.

1 Introducción

- Como parte del desarrollo del proyecto, se llevó a cabo una comparación entre dos enfoques de implementación: uno manual, realizado íntegramente por el equipo, y otro asistido por herramientas de Inteligencia Artificial (vibecoding). El objetivo de este informe es documentar de forma transparente las herramientas, prompts, consumo de recursos y resultados obtenidos en cada enfoque.
- Esta comparación busca evaluar no solo la funcionalidad final alcanzada por cada versión, sino también la eficiencia del proceso de desarrollo: el tiempo insumido, el consumo de recursos de cómputo y el grado de cumplimiento respecto de la rúbrica de funcionalidad definida para la actividad.

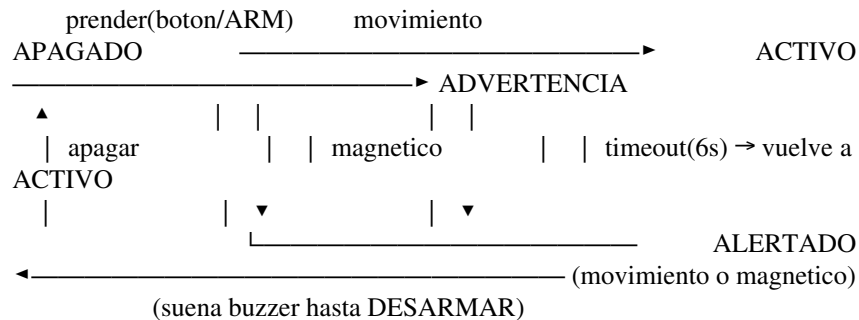
2 Desarrollo

- Herramienta y modelo de IA utilizados: Claude CLI, con Opus 4.8
- Prompt técnico utilizado:
“# Prompt VibeCoding — Proyecto Mochila Antirrobo IoT

Necesito que me generes un proyecto IoT completo para la materia "Sistemas Operativos Avanzados". Es una alarma antirrobo para una mochila/equipaje con tres partes: firmware para ESP32-C3, app Android en Kotlin, e infraestructura local con Docker. Todo funciona local (sin nube ni ngrok).

Qué hace el sistema

Un ESP32-C3 vigila golpes (acelerómetro MPU6050 por I2C) y apertura del cierre (sensor de contacto por ADC, simulado con potenciómetro en Wokwi). Reacciona con un LED y un buzzer siguiendo esta máquina de estados:



Desde una app Android se envían comandos (ARMAR/DESARMAR/PÁNICO) y se ve el estado en tiempo real. Node-RED guarda un historial de activaciones en SQLite y ofrece un panel web.

Embebido (ESP32-C3, PlatformIO + Wokwi)

Requisitos de la cátedra: patrón FSM, mínimo 2 tareas FreeRTOS, sin esperas bloqueantes, sin números mágicos, sensores + actuadores, comunicación WiFi/MQTT bidireccional.

La FSM debe ser C++ puro (sin dependencias de Arduino) para poder testearla en la PC con tests unitarios (Unity framework, entorno `native` de PlatformIO). Usar 3 tareas FreeRTOS (sensores, control con cola de eventos, red con

reconexión) + 1 timer one-shot para la ventana de ADVERTENCIA. ISR del botón con antirrebote por software. Plataforma espressif32@6.9.0 (arduino-esp32 core 2.0.x, usar API LEDC para el buzzer, NO tone/noTone). Incluir el diagram.json para Wokwi.

Tópicos MQTT: `alarma/estado`, `alarma/accel`, `alarma/evento` (ESP32→mundo), `alarma/comando` (mundo→ESP32, acepta "ARM"/"DISARM"/"PANIC"). Broker: usuario `claromio`, password `RiverBest`. WiFi de Wokwi: `Wokwi-GUEST` sin password, broker en `host.wokwi.internal:1883`.

Android (Kotlin, XML layouts)

Requisitos de la cátedra: 2 Activities (NO Compose), un campo de texto + botón, uso de un sensor del teléfono, ejecución en segundo plano (Service), comunicación WiFi/MQTT bidireccional.

Activity 1: campo para IP del broker, botón conectar, botones ARMAR/DESARMAR, muestra estado y aceleración del ESP32. Activity 2: usa acelerómetro del celular para detectar sacudidas → vibra el teléfono + envía PANIC por MQTT. Un Foreground Service mantiene la conexión MQTT viva en segundo plano con notificación persistente. MqttManager como singleton con Eclipse Paho (org.eclipse.paho.client.mqttv3:1.2.5, NO el paho-android-service que está discontinuado). Operaciones de red en hilo de fondo, callbacks al Main Thread con Handler.

Infraestructura Docker

Carpeta `Infraestructura\_VibeCoding\_Extra` con docker-compose.yml:

- **Mosquitto** (eclipse-mosquitto:2): broker MQTT, puerto 1883, autenticación con usuario/password
- **Node-RED** (build con Dockerfile que instala node-red-node-sqlite): suscripto a los tópicos MQTT, guarda historial en SQLite, expone panel web en `/alarma` con estado en tiempo real + botones + tabla de historial, y endpoints REST para estado/historial/comandos
- **InfluxDB + Telegraf** (opcionales, perfil "telemetria"): series de tiempo de la aceleración

Todo en una red Docker compartida. Incluir los archivos de configuración de Mosquitto, flows.json de Node-RED, telegraf.conf, y el Dockerfile.

Documentación

Generar un LEEME_PRIMERO.md con la estructura del proyecto y qué cumple del enunciado, y una GUIA_DE_PRUEBAS.md paso a paso desde cero para

levantar y probar todo (Docker → Wokwi → Android), escrita para alguien sin conocimientos previos.

“

- Cantidad de tokens de IA utilizados: No contamos con el número exacto de tokens utilizados entre entrada y salida, pero en base a las métricas de uso dispuestas por claude estimamos más de 100.000 tokens.
- Tiempo para obtener una versión funcional (Manual vs. Vibecoding): Para el desarrollo de la aplicación manual se precisaron alrededor de 48 horas de trabajo repartidas entre los 5 integrantes. Para el desarrollo de la aplicación con vibecoding se precisaron alrededor de 3 horas de trabajo, principalmente de 1 solo integrante.
- Métricas de CPU y memoria manual

```
=====
Información del dispositivo
=====
Modelo: 2412DPC0AG
Android: 16
Cant. Nucleos: 8
RAM Total: 11312.41 MB

=====
Información de la aplicación
=====
Paquete: com.soal4.mochilaantirrobo
PSS: 160.16 MB
Porcentaje RAM utilizada: 1.4158 %
CPU promedio: 0.00 %

=====
kevin@kevin-Latitude-5421:~/Downloads$
```

- Luego del shake:

```
=====
Información del dispositivo
=====
Modelo: 2412DPC0AG
Android: 16
Cant. Nucleos: 8
RAM Total: 11312.41 MB

=====
Información de la aplicación
=====
Paquete: com.soal4.mochilaantirrobo
PSS: 164.73 MB
Porcentaje RAM utilizada: 1.4562 %
CPU promedio: 0.00 %

=====
kevin@kevin-Latitude-5421:~/Downloads$
```

- Métricas de CPU y memoria vibecoding

```

=====
Información del dispositivo
=====
Modelo: 2412DPC0AG
Android: 16
Cant. Nucleos: 8
RAM Total: 11312.41 MB

=====
Información de la aplicación
=====
Paquete: com.soa.alarma
PSS: 107.34 MB
Porcentaje RAM utilizada: 0.9489 %
CPU promedio: 18.00 %

=====
kevin@kevin-Latitude-5421:~/Downloads$ 

```

- Luego del shake:

```

=====
Información del dispositivo
=====
Modelo: 2412DPC0AG
Android: 16
Cant. Nucleos: 8
RAM Total: 11312.41 MB

=====
Información de la aplicación
=====
Paquete: com.soa.alarma
PSS: 117.80 MB
Porcentaje RAM utilizada: 1.0414 %
CPU promedio: 21.00 %

=====
kevin@kevin-Latitude-5421:~/Downloads$ 

```

- Repositorio Git (código versionado de ambas versiones):
Directorio Manual:
https://github.com/UNLAM-SOA/2026-SOA-Q1-L4/tree/main/Android/Proyecto_sin_Vibecoding
Directorio Vibecoding:
https://github.com/UNLAM-SOA/2026-SOA-Q1-L4/tree/main/Android/Proyecto_con_Vibecoding
- Rúbrica de funcionalidad:

CRITERIOS	SUFICIENTEMENTE LOGRADO (A)	MEDIANAMENTE LOGRADO (B)	INSUFICIENTEMENTE LOGRADO (C)	Puntaje	Eval
FUNCIONA SIN ERRORES	Compila y funciona sin errores.	Funciona con errores menores o <u>warnings</u> .	Presenta errores que impiden su uso.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	A
UTILIZA EJECUCIÓN EN BACKGROUND	Implementa correctamente tareas en <u>background</u> .	Implementa <u>background</u> con algunas fallas.	No implementa tareas en <u>background</u> .	A: Hasta 10 pts. B: Hasta 5 pts. C: 0	B
UTILIZA SENSORES Y PERMISOS DE ANDROID	Utiliza correctamente sensores y permisos.	Presenta problemas menores en sensores o permisos.	Sensores o permisos no funcionan correctamente.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	A
COMUNICACIÓN CON EL EMBEBIDO	Comunicación estable en ambos sentidos.	Comunicación parcial o inestable.	La comunicación falla.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	C
CUMPLIMIENTO DEL OBJETIVO FUNCIONAL	Cumple todos los requisitos funcionales.	Cumple parcialmente los requisitos.	No cumple los requisitos mínimos.	A: Hasta 20 pts. B: Hasta 10 pts. C: 0	B
DISEÑO DE LA APP	Interfaz clara y fácil de usar.	Interfaz funcional con algunos problemas.	Interfaz deficiente o difícil de usar.	A: Hasta 10 pts. B: Hasta 5 pts. C: 0	B
TOTAL				100	60

3 Conclusiones

A partir de la comparación entre el desarrollo manual y el desarrollo asistido por Inteligencia Artificial, se pudieron extraer las siguientes conclusiones:

Notamos una clara ventaja en los tiempos de desarrollo de la app utilizando técnicas de vibecoding, sin embargo, los usos de CPU son mucho más intensivos que los de la app desarrollada de forma manual. También encontramos que el código es menos legible y requirió de un conjunto de iteraciones para lograr que funcione según lo esperado.

El vibecoding desmedido o sin supervisión puede ocasionar problemas de rendimiento importantes, o inconsistencias grandes entre el resultado esperado y el obtenido. También cabe aclarar que consideramos, es una herramienta que bien utilizada demuestra una clara mejora de rendimiento en el tiempo requerido para el desarrollo de software.

4 Referencias

1. Anthropic: Claude Code. <https://docs.claude.com/en/docs/claude-code/overview> (2026)
1. Wikipedia: Vibe coding. https://en.wikipedia.org/wiki/Vibe_coding (2026)
2. Anthropic: Introducing Claude Opus 4.8. <https://www.anthropic.com/news/claude-opus-4-8> (2026)